

# LUT core-module ablation: three generations of lookup math

## Comparison table

2026-09-13

### Protocol

**Architecture:** LUT transformer,  $E = 384$ , 6 layers, 6 attention heads. FFN = `CompressionMultiHeadLUT` with  $H = 4$  heads  $\times$   $tph = 128$  tables ( $P = H \cdot tph = 512$  tables per token),  $d_{in} = d_{out} = 48$ ,  $n = 8$  anchor pairs per table ( $K = 2^n = 256$  cells); `compress`  $384 \rightarrow 192$ , `decompress`  $192 \rightarrow 384$ . **Training:** 16,000 steps, batch size 48 (24576 tokens), untied embeddings, LUT tables without weight decay. **Eval:** `evaluate_bpb_fixed`, 48 rows  $\times$  100 steps, first 12 rows skipped.

### Counting convention

- Core LUT only, per token, one layer. The `compress/decompress` projections are identical in every row and excluded (147,456 MACs forward).
- **FLOP** = one scalar float operation ( $+$ ,  $-$ ,  $\times$ ,  $\div$ , a comparison,  $|\cdot|$ ,  $\exp$ ,  $\log \sigma$ ,  $\sigma$ ). **MAC** = a multiply accumulated into a sum; 1 MAC = 2 FLOPs, included in the FLOP column. Memory reads and integer work are not counted.
- **Forward** = inference. **Backward** = all additional training arithmetic, including per-table gradients of per-layer scalars.

## Table A — forward function and gradient function

Per variant, (a) is the function computed on the forward pass and (b) the function whose gradient training uses: the weights always receive the gradient of (a), while the input, the temperatures and the learned per-layer scalars receive the gradient of (b) with the table weights held constant.

| Variant                                                                         | (a) Forward: the function computed                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | (b) Backward: the function whose gradient is used                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>GEN 1 — Spiking Manifesto basic model</b>                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 1.1 Hard forward, two-alternatives backward                                     | $y = \sum_t W_t[c_t]$ , summed over tables $t$ : each of table $t$ 's $n$ anchor pairs gives a signed margin $u_j = z[a_j] - z[b_j]$ ; the $n$ signs $[u_j > 0]$ are the bits of the binary index $c_t \in \{0, \dots, 2^n - 1\}$ of the row read from $W_t \in \mathbb{R}^{K \times d}$ , $K = 2^n$ .                                                                                                                                                                                                                                                                                                                                                                                       | The 2-cell blend $\sum_t [(1 - U_t)W_t[c_t] + U_tW_t[c'_t]]$ , $U_t = U(u_{j^*})$ , $U(u) = 0.5/(1 +  u )$ , with the least-confident pair $j^* = \arg \min_j  u_j $ and $c'$ ( $c$ with bit $j^*$ flipped) held fixed. The pairing with (a) is exact as a stop-gradient composite; the blend is $C^1$ across a bit flip and jumps only where $j^*$ switches.                                                                                                                                                                                                                                                         |
| 1.2 Soft forward, two-alternatives backward                                     | (a) = (b): the blend of 1.1(b), applied at eval too.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>GEN 2 — FastMHL math</b>                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 2.1 Hard forward, all-alternatives backward (all $K$ cells)                     | $y = \sum_t W_t[c_t]$ , as 1.1(a).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | With $b_j(\ell) = +1$ if the bit of $\ell$ for pair $j$ is 1, else $-1$ , and the squashed margin $\hat{m}_j = u_j/(T_s +  u_j ) \in (-1, 1)$ : $s_\ell = \sum_j b_j(\ell) \hat{m}_j$ scores how well row $\ell$ 's bit pattern agrees with the actual signs of the margins (small margins count little either way). All $K$ rows of each table are read, softmax-weighted within that table: $\sum_t \sum_{\ell < K} \text{softmax}_\ell(s_\ell/T_{sel}) W_t[\ell]$ . $T_s, T_{sel}$ : per-layer learned scalars. Globally $C^1$ in the input; never uses the address, whose row is the one with the largest weight. |
| 2.2 Hybrid_smooth two-alternatives forward, all-alternatives backward           | Reads two cells, the addressed $c_t$ and the alternative $c'_t$ ( $c_t$ with the bit of the least-confident pair $j^* = \arg \min_j  u_j $ flipped), blended by a weight set by that pair's margin: $y = \sum_t [(1 - w_t)W_t[c_t] + w_tW_t[c'_t]]$ , $w_t = \sigma(-2\rho_{j^*}/T_{sel}) \in (0, \frac{1}{2}]$ , $\rho_j =  u_j /(T_s +  u_j )$ , $\sigma$ the logistic sigmoid; applied at eval too.                                                                                                                                                                                                                                                                                       | The same softmax-weighted read of all $K$ rows as 2.1(b), with the same $T_s, T_{sel}$ parameters as (a).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 2.3 Hard forward, 2-alternatives backward                                       | The hard read of 1.1(a).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | The softmax of 2.1(b) restricted to the two cells $c_t$ and $c'_t$ ( $c_t$ with the bit of the least-confident pair $j^* = \arg \min_j  u_j $ flipped) and renormalised over that pair. The weight on $c'_t$ is $\sigma(-2\rho_{j^*}/T_{sel})$ , $\rho_j =  u_j /(T_s +  u_j )$ , $\sigma$ the logistic sigmoid: exactly 2.2(a)'s $w_t$ , so this is 2.2(a)'s 2-cell blend.                                                                                                                                                                                                                                           |
| 2.4 Hybrid_smooth two-alternatives forward, 2-alternatives backward             | (a) = (b): the 2-cell blend of 2.2(a). The backward keeps only 2 of the $K$ rows, the addressed $c_t$ and its least-confident neighbour $c'_t$ , which are exactly the two cells the forward reads, so the restricted softmax equals the forward weight $w_t$ and forward and backward coincide; unlike 2.2, the input gradient reaches only pair $j^*$ .                                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>GEN 3 — LightMHL math (LookupFFN-inspired): learned margin score</b>         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 3.1 The Gen-3 form: learned margin score, one cell, per-layer $(\beta, \gamma)$ | (a) = (b): $y = \sum_t s_t W_t[c_t]$ with the address $c_t$ held fixed; $s_t = S_t P_t^\gamma$ , $S_t = \sum_j m_j$ , $P_t = \prod_j \sigma(\beta m_j)$ , $m_j =  u_j $ , computed as $S_t \exp(\gamma \sum_j \log \sigma(\beta m_j))$ ; $\beta, \gamma$ learned per layer from (2, 1), where $s_t$ is LookupFFN's margin score. Discontinuous at cell boundaries, since $\sigma(0) > 0$ (trained jump: median 2.22% of $\ y\ $ ).                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 3.2 The Gen-3 form, two alternatives, per-layer $(\beta, \gamma, \tau)$         | (a) = (b): each table reads two cells, $y = \sum_t s_t [(1 - v_t)W_t[c_t] + v_tW_t[c'_t]]$ , with $s_t$ as in 3.1 over all $n$ margins and $c'_t$ the cell $c_t$ with the bit of the smallest margin $j^* = \arg \min_j m_j$ flipped ( $c_t, c'_t, j^*$ held fixed). $v_t$ is a softmax over the two cells with logits 0 and $-2m_{j^*}/\tau$ (flipping bit $j$ lowers $\langle u, b \rangle$ by $2m_j$ ), i.e. $v_t = \sigma(-2m_{j^*}/\tau) \in (0, \frac{1}{2}]$ ; $\tau$ learned per layer from 0.5. The input gradient reaches all $n$ margins through $s_t$ and $m_{j^*}$ alone through $v_t$ . Continuous across a bit flip ( $v_t = \frac{1}{2}$ there); jumps where $j^*$ switches. |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

*continued on next page*

| Variant                                           | (a) Forward: the function computed                                                                        | (b) Backward: the function whose gradient is used                                                                                                                                                                                                                                                                                                         |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3.3 Hard forward, 3.1 backward (one cell)         | The hard read of 1.1(a): $y = \sum_t W_t[c_t]$ , not scaled by a score; nothing else is computed at eval. | 3.1's $\sum_t s_t W_t[c_t]$ , $s_t = S_t P_t^\gamma$ , $S_t = \sum_j m_j$ , $P_t = \prod_j \sigma(\beta m_j)$ , $m_j =  u_j $ , $\sigma$ the logistic sigmoid, $\beta, \gamma$ learned per layer. The input, $\beta$ and $\gamma$ get its gradient; the weights get (a)'s unscaled gradient at $W_t[c_t]$ .                                               |
| 3.4 Hard forward, 3.2 backward (two alternatives) | The hard read of 1.1(a), as 3.3(a).                                                                       | 3.2's $\sum_t s_t [(1 - v_t)W_t[c_t] + v_t W_t[c'_t]]$ , $s_t$ as in 3.3(b), $c'_t$ the cell $c_t$ with the bit of the smallest margin $j^* = \arg \min_j m_j$ flipped, $v_t = \sigma(-2m_{j^*}/\tau)$ , $\tau$ learned per layer. The input, $\beta$ , $\gamma$ and $\tau$ get its gradient; the weights get (a)'s unscaled gradient at $W_t[c_t]$ only. |

**Table B — cost and results**

Per token, one layer, core LUT only;  $P = 512$ ,  $n = 8$ ,  $K = 256$ ,  $d = 48$ .

| Variant | Forward FLOPs                                             | Backward FLOPs                                                       | Forward MACs                   | Backward MACs                           | Validation bpb                   | Validation bpb, +TV ( $\lambda = 10$ ) | Implementation (class used)                                                                                                                         |
|---------|-----------------------------------------------------------|----------------------------------------------------------------------|--------------------------------|-----------------------------------------|----------------------------------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.1     | $P(2n + d) = \mathbf{32,768}$                             | $P(5d + 2n + 9) = \mathbf{135,680}$                                  | 0                              | $P \cdot 2d = \mathbf{49,152}$          | <b>TBD — not yet implemented</b> | <b>TBD — not yet implemented</b>       | MultiHeadLut(smooth_mode=False, n_alternatives=1, uncertainty_mode=INVERSE_L1); not wired into CompressionMultiHeadLUT.                             |
| 1.2     | $P(4d + 4n + 4) = \mathbf{116,736}$                       | $P(8d + 9) = \mathbf{201,216}$                                       | $P \cdot 2d = \mathbf{49,152}$ | $P \cdot 4d = \mathbf{98,304}$          | <b>TBD — not yet implemented</b> | <b>TBD — not yet implemented</b>       | MultiHeadLut(smooth_mode=True, n_alternatives=1); not wired in.                                                                                     |
| 2.1     | $P(2n + d) = \mathbf{32,768}$                             | $P(2Kd + 4nK + 9K + 11n + d) = \mathbf{18,026,496^a}$                | 0                              | $P(Kd + 2nK) = \mathbf{8,388,608}$      | <b>1.180622</b> (exp_n_0121)     | <b>TBD — TV not wired for Gen 2</b>    | FastMultiHeadLut(forward_mode='hard', backward_topk=0).                                                                                             |
| 2.2     | $P(2kd + 4n + 5) = \mathbf{117,248}$ , $k = 2$ cells read | $P(2Kd + 4nK + 9K + 11n + 2kd) = \mathbf{18,100,224^a}$              | $Pkd = \mathbf{49,152}$        | $P(Kd + 2nK + kd) = \mathbf{8,437,760}$ | <b>TBD — not yet run</b>         | <b>TBD — TV not wired for Gen 2</b>    | FastMultiHeadLut(forward_mode='hybrid_smooth', backward_topk=0).                                                                                    |
| 2.3     | $P(2n + d) = \mathbf{32,768}$                             | $P(2kd + 4nk + 9k + 12n + d - 1) = \mathbf{213,504}$ , $k = 2$ cells | 0                              | $P(kd + 2nk) = \mathbf{65,536}$         | <b>TBD — not yet run</b>         | <b>TBD — TV not wired for Gen 2</b>    | FastMultiHeadLut(forward_mode='hard', backward_topk=1); backward_topk counts the cells kept besides $c_t$ .                                         |
| 2.4     | $P(2kd + 4n + 5) = \mathbf{117,248}$                      | $P(4kd + 9n + 15) = \mathbf{241,152}$                                | $Pkd = \mathbf{49,152}$        | $P2kd = \mathbf{98,304}$                | <b>TBD — not yet run</b>         | <b>TBD — TV not wired for Gen 2</b>    | FastMultiHeadLut(forward_mode='hybrid_smooth', backward_topk=1).                                                                                    |
| 3.1     | $P(2kd + 7n + 1) = \mathbf{78,336}$ , $k = 1^b$           | $P(4kd + 10n + 8) = \mathbf{143,360^b}$                              | $Pkd = \mathbf{24,576}$        | $P2kd = \mathbf{49,152}$                | <b>1.169328</b> (exp_g_0248)     | <b>1.163698</b> (exp_g_0249)           | LightMultiHeadLUT(confidence_form='learned_margin', learned_margin_init=(0, 2, 1), learned_margin_freeze_g=True), with $g$ the log-gain, held at 0. |
| 3.2     | $P(2kd + 8n + 8) = \mathbf{135,168}$ , $k = 2^b$          | $P(4kd + 10n + 23) = \mathbf{249,344^b}$                             | $Pkd = \mathbf{49,152}$        | $P2kd = \mathbf{98,304}$                | <b>TBD — not yet run</b>         | <b>TBD — not yet run</b>               | As 3.1 with read_top_n=2, read_tau=0.5, read_tau_learnable=True.                                                                                    |
| 3.3     | $P(2n + d) = \mathbf{32,768}$                             | $P(3d + 15n + 9) = \mathbf{139,776^b}$                               | 0                              | $Pd = \mathbf{24,576}$                  | <b>TBD — not yet implemented</b> | <b>TBD — not yet implemented</b>       | LightMultiHeadLUT has no hard forward (every path returns the $s_t$ -scaled read); would be 3.1's config plus one.                                  |
| 3.4     | $P(2n + d) = \mathbf{32,768}$                             | $P(5d + 16n + 29) = \mathbf{203,264^b}$                              | 0                              | $P2d = \mathbf{49,152}$                 | <b>TBD — not yet implemented</b> | <b>TBD — not yet implemented</b>       | As 3.3, with 3.2's config.                                                                                                                          |

## Notes to Table B.

- <sup>a</sup> Dominated by  $2Kd$ : every table's  $K$  rows are read; several [tokens, 512, 256] fp32 buffers of about 12.9 GiB at 24,576 tokens.
- <sup>b</sup> Per table, forward:  $n$  comparisons,  $n$  differences,  $n$  absolute values,  $n$  products  $\beta m_j$ ,  $n \log \sigma$ , the two sums,  $\times \gamma$ , exp and the final product  $(7n+1)$ , plus the scaled rows  $(2kd)$ ; 3.2 adds the argmin  $(n-1)$ , the logit  $-2m_{j^*}/\tau$  (2), the two-way softmax (4) and  $s_t$  times the two weights (2). Backward: the row terms  $(4kd)$ , the input term  $(8n+3)$  and the per-table accumulation of the parameter gradients  $(\partial \log \gamma \ 3, \partial \log \beta \ 2n+2)$ ; 3.2 adds  $\partial s_t$  from two rows (3),  $\partial v_t$  (2), the softmax backward (5),  $\partial m_{j^*}$  (3) and  $\partial \log \tau$  (2). The parameter gradients are scalar multiplies, so they add no MACs. 3.3, 3.4: the forward is the hard read, so the score  $(5n+1)$ , all of 3.1's forward but the comparisons and differences) and, for 3.4, the argmin, logit and softmax  $(n+5)$  are training-only and join 3.1's / 3.2's backward, which swaps each row's score-scaled weight update  $(2d, d \text{ MACs})$  for one plain update at  $c_t$  ( $d$ ):  $3d+15n+9$  and  $5d+16n+29$ .

## Cell TV regularisation (Table B's +TV column)

- **Penalty:**  $\lambda \cdot \frac{1}{6} \sum_{\text{layers}} \frac{1}{P n 2^{n-1}} \sum_t \sum_{(c,c')} \|W_t[c] - W_t[c']\|^2$ , squared L2 over the cell pairs  $(c, c')$  that differ in one bit,  $\lambda = 10$ ; its gradient is added once per step, before gradient clipping. Training only: +6,144 FLOPs and +2,048 MACs per token per layer (amortised over 24,576 tokens); inference unchanged.
- **Runnable for Gen 3 only:** the trainer's penalty (`lut_tv_penalty`) collects `LightMultiHeadLUT` tables only and is a constant zero on Gen 1 and Gen 2 models, so their +TV cells need it extended to `FastMultiHeadLut` and `MultiHeadLut` tables first.

## Remaining work

- **Run 2.2, 2.3 and 2.4:** fork `exp_n_0121` verbatim, changing only `lut_forward_mode` / `lut_backward_topk`. No matching run exists (earlier `hybrid_smooth` runs are  $n = 6$ , tied, old eval; `exp_n_0188` is  $n = 7$  with a gate on).
- **Run 3.2:** fork `exp_g_0248`, adding `lut_read_top_n: 2`, `lut_read_tau: 0.5`, `lut_read_tau_learnable: true`. No learned-score  $n = 2$  run exists; `exp_g_0195` is  $n = 2$  with the plain margin score, not this form. Its +TV twin adds `lut_cell_smoothness: 10`.
- **Wire TV into Gen 1 and Gen 2:** extend `lut_tv_penalty` beyond `LightMultiHeadLUT`; until then their +TV cells cannot be run.
- **Implement 1.1, 1.2:** wire Gen 1 into `CompressionMultiHeadLUT`, which accepts only `lut_impl`  $\in \{\text{fast, light, bh4}\}$  (`compression_mhl.py:190`), as `MultiHeadLut` with `anchor_candidates` [128, 4, 48].
- **Implement 3.3, 3.4:** give `LightMultiHeadLUT` a hard forward (today every path, the native eval kernel included, returns the  $s_t$ -scaled read): the plain read plus  $f - \text{sg}(f)$ ,  $f$  being 3.1's or 3.2's read with the table weights held constant and  $\text{sg}$  the stop-gradient; plumb it through `CompressionMultiHeadLUT` and `model_build`.
- **Seeds:** zero replicates at this geometry; one extra seed of `exp_g_0248` ( $\approx 0.95$  h) would give a real LUT seed spread. Reference: vanilla dense FFN 1.1651 / 1.1618 at 16K (two seeds), 1.1154 at 48K.